

This uses the helper function `drain`, which just ignores all output of a `Process`:

```
final def drain[O2]: Process[F,O2] = this match {
  case Halt(e) => Halt(e)
  case Emit(h, t) => t.drain
  case Await(req,recv) => Await(req, recv andThen (_.drain))
}
```

When run via `runLog`, `converter` will open the input file and the output file and incrementally transform the input stream, ignoring commented lines.

15.3.6 Effectful channels

We can generalize to allow responses other than `Unit`. The implementation is identical! It turns out that the operation just has a more general type than we gave it. Let's call the more general operation `through`:

```
def through[O2](p2: Process[F, O => Process[F,O2]]): Process[F,O2] =
  join { (this zipWith p2)((o,f) => f(o)) }
```

Let's introduce a type alias for this pattern:

```
type Channel[F[_],I,O] = Process[F, I => Process[F,O]]
```

`Channel` is useful when a pure pipeline must execute some I/O action as one of its stages. A typical example might be an application that needs to execute database queries. It would be nice if our database queries could return a `Process[IO,Row]`, where `Row` is some representation of a database row. This would allow the program to process the result set of a query using all the fancy stream transducers we've built up so far.

Here's a signature for a simple query executor, which uses `Map[String,Any]` as the (untyped) row representation (see the chapter code for the implementation):

```
import java.sql.{Connection, PreparedStatement, ResultSet}

def query(conn: IO[Connection]):
  Channel[IO, Connection => PreparedStatement, Map[String,Any]]
```

We could certainly write a `Channel[PreparedStatement, Source[Map[String,Any]]]`, so why don't we? Because we don't want code that uses our `Channel` to have to worry about how to obtain a `Connection` (which is needed to build a `PreparedStatement`). That dependency is managed entirely by the `Channel` itself, which also takes care of closing the connection when it's finished executing queries.

15.3.7 Dynamic resource allocation

Realistic programs may need to allocate resources dynamically, while transforming some input stream. For example, we may encounter scenarios like the following:

- *Dynamic resource allocation*—Read a file, `fahrenheits.txt`, containing a list of filenames. Concatenate these files into a single logical stream, convert this stream to Celsius, and output the joined stream to `celsius.txt`.